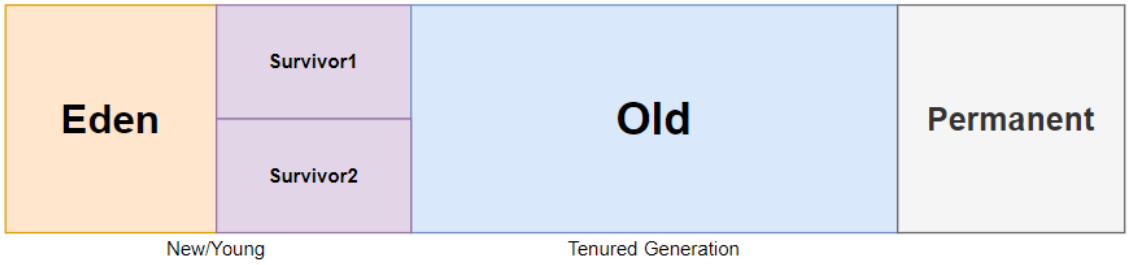
1. Java8 이전까지의 Heap 메모리 구조



heap 메모리 영역은 크게 3 개의 영역으로 분리되어 있다.

1. New/Young

- 일단 메모리에 객체가 생성되면 `Eden` 영역에 객체가 지정된다.
- Eden 영역에 데이터가 가득 차면, Eden 영역에 있던 객체가 Survivor1 또는 Survivor2 로 옮겨진다. 대부분의 JVM 메모리 구조를 설명하는 그림에서 1 과 2 로 나누는데, 이 두 개의 영역 사이에 우선순위가 있는 것은 아니다. 단지 쉽게 설명하기 위해 1 과 2 로 나눈다고 생각하면 된다.
- Eden 영역에서 Survivor1 또는 Survivor2 로 옮겨지는 객체들은 어딘가에서 참조되고 있는 객체들이다. 둘 중 하나의 영역이 가득 차게 되면 공간이 남아있는 Survivor 로 이동하게 된다.
- **이러한 매커니즘 때문에 Survivor1 또는 Survivor2 둘 중 하나는 항상 비워있는 공간이 있는 채로 유지된다.**

이러한 과정에서 1 차 GC 라고 불리는 `Minor GC` 가 발생한다. Minor GC 는 New/Young 영역에서 발생하는 GC 로 Eden 영역 또는 Survivor1 또는 Survivor2 에서 사용되지 않는 객체들을 삭제한다.

2. Old

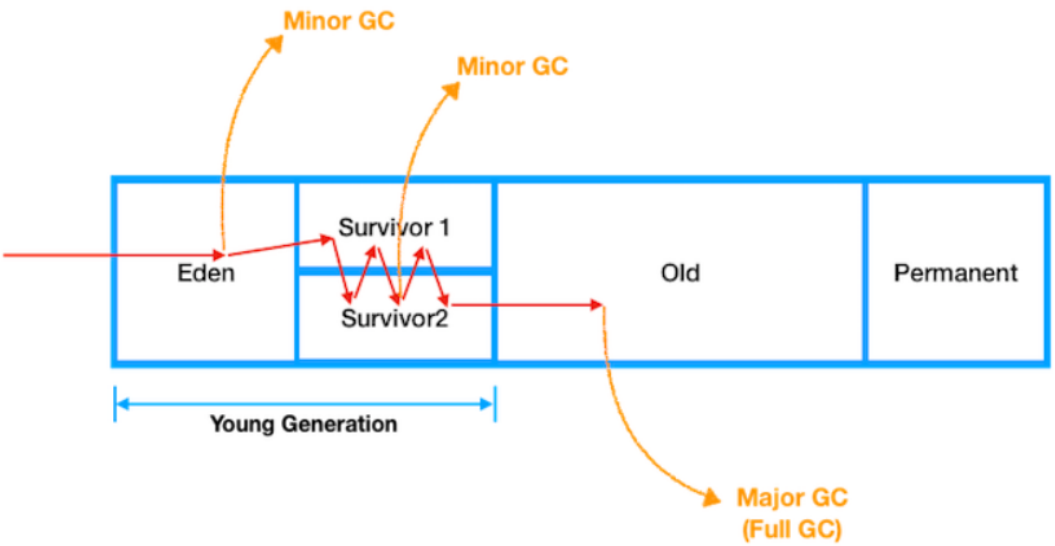
- Survivor1 / Survivor2 를 왔다 갔다 하는 과정에서 오랫동안 살아남은 객체들은 Old 영역으로 이동한다. 보통 Old 영역은 **Young 영역보다 크게 할당하며, 이러한 이유로 Old 영역의 GC 는 Young 영역보다 적게 발생한다.**
- 단, Young 영역에서 Old 영역으로 넘어가는 객체 중, Survivor 영역을 거치지 않고 Eden 영역에서 바로 Old 영역으로 넘어가는 객체도 존재하는데, 이는 **객체의 크기가 아주 클 경우** 발생한다. 예를들어 Survivor 영역의 크기가 16MB 인데 객체의 크기가 20MB 일 경우에 해당한다.

오랫동안 살아남은 객체들이라면, 얼마나 오랫동안일까?

오래되었다고 하는 기준은 Young Generation 영역에서 Minor GC 가 발생하는 동안 얼마나 오래 살아남았는지로 판단한다. 각 객체는 Minor GC 에서 살아남은 횟수를 기록하는 ***age bit*** *를 가지고 있으며,* ***Minor GC 가 발생할 때마다 age bit 값은 1 씩 증가*** *하게되며, age bit 값이* ***MaxTenuringThreshold 라는 설정값을 초과하게 되는 경우 Old Generation 영역을 객체가 이동*** *되는 것이다. 또는 Age bit 가 MaxTenuringThreshold 초과하기 전이라도 Survivor 영역의 메모리가 부족할 경우에는 미리 Old Generation 으로 객체가 옮겨질 수도 있다.*

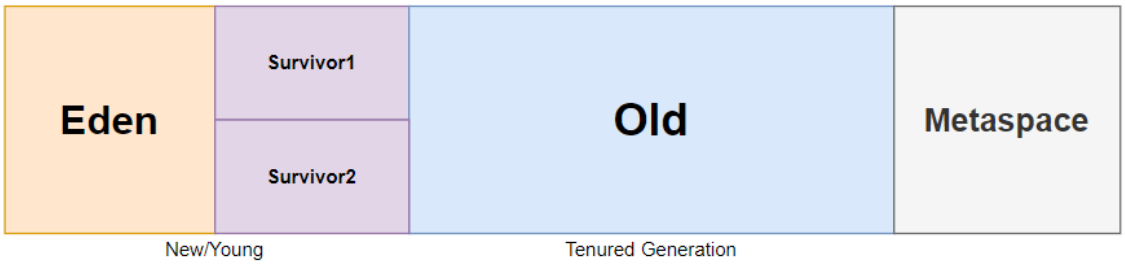
Old 영역에서는 2 차 GC 인 `Major GC(FULL GC)`가 일어나게 되며 GC 를 진행하는 Thread 를 제외하고 이외의 모든 Thread 를 멈춘 상태로 GC 가 진행된다. 이와 같이 GC 를 진행하는 Thread 이외에 모든 Thread 를 멈추는 상태를 `Stop-the-world` 라고 하며, **어떠한 GC 알고리즘을 사용하더라고 Stop-the-World 상태를 피할수는 없다.**

JVM 에 대해서 자세히 공부하고 있다면 GC 튜닝에 대해서 들어본 적이 있을것이다. GC 를 튜닝하는 이유가 바로 **Stop-the-world** 의 시간을 최소한으로 줄이기 위함이다.



heap 메모리 영역의 흐름을 가장 잘 표현한 그림

2. Java8 이후의 Heap 메모리 구조



위에서 Heap 메모리 구조를 설명하면서 설명하지 않은 영역이 하나 있다. 바로 **Permanent** 영역이다.
Permanent 영역의 경우 Java8 부터 Metaspace 영역으로 변경되었다. 기존의(자바 8 이전) Permanet 영역에는 다음과 같은 정보들이 저장되었다.

- Class 의 Meta Data
- Method 의 Meta Data
- **Static Object 변수, 상수**
- JVM, JIT 관련 데이터 등

이러한 데이터들을 저장하던 Permanent 영역을 자바 8 에서부터는 Metaspace 라는 영역으로 대체하였고, 이 Metaspace 영역은 Native 메모리 영역으로 JVM 이 아닌 OS 에 의해 관리되도록 변경되었다.
즉, 자바 8 이후부터 static 객체는 heap 영역이 아닌 별도의 네이티브 메모리 영역에서 관리된다.

거의 그럴일은 없지만 가끔씩 Collection 객체를 static 하게 구현하여 값을 계속해서 추가하다가 Perm 영역이 가득차서 `OutOfmemoryerror permgen space` 라는 Error 를 경험한 적이 있을 것이다.

```
static List<Object> list = new ArrayList<>();
```

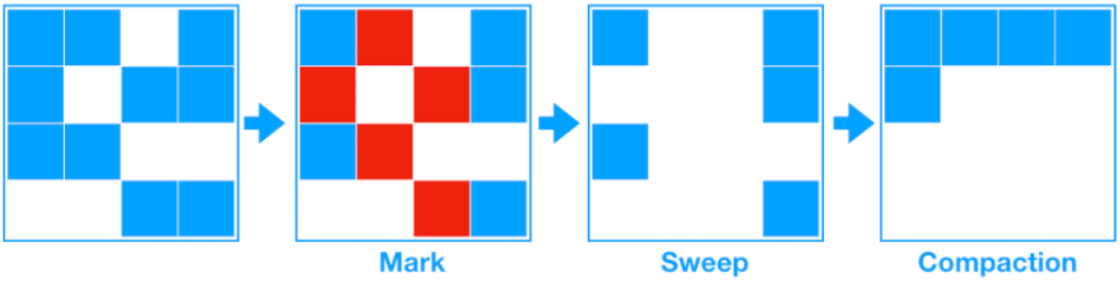
즉, 이러한 OOM 에러가 발생하는 현상을 개선하기 위해 기존에 Perm 영역에 저장되던 static Obejct 의 변수와, 상수화된 static Object 를 Heap 영역으로 이동시켜 GC 의 대상이 되도록 변경하고, 메타데이터 정보들을 OS 가 관리하는 영역으로 옮겨 Perm 영역의 사이즈 제한을 없앤 것이라고 할 수 있다.

GC 알고리즘 종류

1. Serial GC

JDK 5,6 에서 사용되는 GC 로 Minor GC 와 Major GC 모두 싱글스레드로 수행하기 때문에 GC 가 진행되는 시간(Stop-The-World)이 다른 GC 에 비해 오래걸린다.
해당 GC 는 응답속도를 신경쓸 필요가 없는 애플리케이션에서 사용할 수 있지만, 더 좋은 GC 방법이 있기 때문에 사용하지 않는것이 바람직하다. 몇몇 책에서는 절대 사용하면 안되는 GC 로 설명하고 있다.
Mark-Sweep-Compaction 알고리즘을 이용한다.

Mark-Sweep-Compaction 알고리즘
1. 사용되지 않는 객체를 식별(Mark)
2. 식별한 객체를 삭제(Sweep)
3. 파편화된 메모리를 정리/압축



mark – sweep – compaction

2. Parallel GC

Young 영역에서 발생하는 Minor GC 수행시 멀티 스레드를 사용한다. 따라서 Minor GC 수행시 Serial GC 보다 수행시간이 빠르다. 만약 본인 장비의 CPU 가 단일코어라면 Parallel GC 로 설정해도, Serial GC 로 수행된다.
Mark-Sweep-Compaction 알고리즘을 이용한다.

3. Parallel old GC

Parallel GC 를 조금 업그레이드한 버전이다. Old 영역에서 발생하는 Full GC 도 병렬로 처리하며, Old 영역에서 발생하는 GC 를 처리할때 Mark-Summary-Compaction 알고리즘을 이용한다.
기존의 Sweep 대신에 있는 Summary 단계에서는 앞서 GC 를 수행한 영역에대해서 별도로 살아있는 객체를 식별한다는 점에서 다르고 더 복잡한 방식을 사용한다.

4. CMS(Concurrent Mark & Sweep) GC

CMS GC 는 Old 영역에서 발생하는 Full GC 의 수행시간을 최소한으로 하는데 초점을 둔 GC 방식이다. 즉 Stop-The-World 의 시간을 최소화 한다.
Full GC 가 수행되는 시간을 최소한으로 줄이기 위해, GC 의 대상 객체를 최대한 정밀하게 파악한다.

- Initial Mark : 현재 살아남은 객체를 탐색하는데, GC ROOT 에서 참조하는 객체들만 우선적으로 탐색하기 때문에 STW 발생 시간이 매우 적다. (STW 발생)

- Concurrent Mark : Initial Mark 에서 탐색한 객체들이 참조하고 있는 객체를 찾아가며, GC 의 대상인지 판별한다. (STW 발생하지 않음)
- ReMark : Concurrent Mark 과정 중 새로 생성된 객체나, 참조자 끊기는 등 변경된 객체가 있는지 다시한번 검사한다. (STW 발생 - 멀티스레드로 동작하기 때문에 STW 시간이 짧다.)
- Concurrent Sweep : Remark 단계까지 검증이 완료된 GC 대상 객체들을 삭제한다. (STW 없이 진행된다)

이러한 과정을 거치기 때문에 Full GC 가 수행되는 시간을 최소한으로 줄일 수 있게 된다.
하지만 CMS GC 는 Compaction 작업을 기본적으로 진행하지 않기 때문에 메모리 단편화에 대한 문제를 신경 써야 한다.
만약 연속적인 메모리 할당이 불가능할 정도로 메모리 단편화가 진행되었다면 Compaction 작업을 수행해야 하는데, 해당 작업은 다른 GC 에서의 Compaction 작업에 비해 Stop-the-world 시간이 길다. 따라서 CMS GC 를 사용할 때는 Compaction 작업이 얼마나 일어나는지 파악한 후 사용하는 것이 좋다.

5. G1 GC(Garbage First GC)

G1 GC 는 기존 GC 알고리즘 들로는 큰 메모리에서 좋은 성능을 내기 힘들었기 때문에 이를 개선하고자 등장했다.
해당 GC 알고리즘에서는 그동안 알고있던 Heap 메모리 구조를 잠시 잊어 둘 필요가 있다.
힙 메모리 영역 전체를 Region 이라는 논리적인 단위로 나눠서 관리하며, 이렇게 나뉜 Region 에 특정 역할(Eden, Survivor, Old 등)을 동적으로 부여한다.

Region 이라는 논리적인 단위로 메모리를 관리하여 CMS 와 달리 Compaction 단계를 진행하고 메모리 단편화 문제를 없앴다. 또한 STW 의 시간을 예측할 수 있다는 것이 G1 GC 의 큰 장점 중 하나이다.

위 그림과 같이 Region 단위로 나뉘는데, JVM 힙은 2048 개의 Region 으로 나뉠 수 있으며, 각 Region 의 크기는 1MB ~ 32MB 사이로 지정될 수 있다.
Eden, Survivor, Old 는 이전에도 동일하게 존재했던 영역이지만 Humonogous 와 Avaliable/Unused 는 처음보는 영역이다.

- Humonogous : Region 크기의 50%를 초과하는 객체가 저장되는 공간이며, 이 공간에서는 GC 가 효율적으로 일어나지 않는다.
- Avaliable/Unused : 아직 사용되지 않은, 비어있는 공간의 Region 이다.

G1 GC 에서 **Young GC** 를 수행할 때는 STW(Stop-The-World) 현상이 발생하며, STW 시간을 최대한 줄이기 위해 멀티스레드로 GC 를 수행한다. **Young GC 는 각 Region 중 GC 대상 객체가 가장 많은 Region(Eden 또는 Survivor 역할) 에서 수행** 되며, 이 Region 에서 **살아남은 객체를 다른 Region(Survivor 역할) 으로 옮긴 후, 비워진 Region 을 사용가능한 Region 으로 돌리는 형태** 로 동작한다.

G1 GC 의 Old GC 는 CMS GC 처럼 GC 수행 시간을 줄이는데 초점을 둔다. CMS 와 다른 점은 살아있는 객체를 찾아서 Region 에 옮긴 후 남은 객체들을 삭제하는 방식으로 GC 가 진행된다.